

Machine Learning Based Detection of Malware in MSI File

Guy Levy 322317918, Raphael Coeffic 337614747

2025 / 2026

1 Abstract

This research aims to develop a Machine Learning based system capable of automatically detecting malicious software embedded within installation files. Traditional malware detection approaches rely heavily on static signatures and often fail to identify new or obfuscated threats. Machine Learning provides a suitable alternative, as it can learn complex patterns from large datasets and generalize to previously unseen malware variants without requiring manually crafted rules.

The study explores a data-driven detection approach in which structural and behavioral features extracted from installation packages are used to train classification models. As a focused case study, the research investigates Microsoft Installer (MSI) files, a format increasingly exploited by attackers due to its trusted nature and reduced visibility in conventional detection pipelines.

The expected contribution of this work is the development of an automated, scalable, and efficient detection framework that improves accuracy relative to signature-based methods, reduces false positives, and provides new insights into feature patterns characteristic of malicious installers.

2 Introduction

In recent years, the increasing sophistication of malware distribution methods has presented significant challenges to cybersecurity defenses. One emerging threat is the abuse of Microsoft Installer (MSI) files; a filetype created by Microsoft to install whole programs using a simple installer.

Attackers have learned to exploit the trustworthy nature of the file, seeming "official" and "trustworthy" to the regular eye. Attackers embed malicious code and scripts in the installed packages, which we will discuss later on in our project.

The central issue this paper addresses is the use of MSI files as a covert delivery mechanism for malware. Because MSI files often require administrative permissions and are signed or distributed through seemingly legitimate channels, they provide an attractive opportunity for attackers to bypass user suspicion and system security measures. As a result, even experienced users and well-protected organizations may unknowingly execute harmful payloads disguised as legitimate software.

Understanding how MSI based malware operates is critical to improving both prevention and detection strategies. The subject is especially important in enterprise environments, where software deployment at scale often relies on automated MSI distribution systems that can amplify the impact of a single compromised installer.

For our view, understanding how attackers exploit .MSI is critical to understanding how attackers evolve to operate through embedded, "legitimate" operating system software to exploit their victims.

The purpose of this research is to analyze the mechanisms by which malware can be embedded in .MSI files, evaluate the techniques used for detection, and propose improved methods for identifying malicious installer behavior. By investigating both technical and procedural vulnerabilities in the MSI, this study aims to contribute to a deeper understanding of the risks associated with installer-based malware and to outline practical recommendations for the operating system clients.

3 A comprehensive review of literature in the field

3.1 Malware Detection in PE files using Machine Learning, By Samarth Tyagi

This article proposes an efficient malware detection method based on the static analysis of Portable Executable (PE) files using supervised Machine Learning (ML) classifiers. Recognizing the limitations of traditional signature-based detection against modern threats, the authors leveraged 54 structural

features extracted from PE headers, subsequently reduced to 13 highly discriminative features, including entropy metrics. A large dataset of over 137,000 PE files was used to train and validate several models, including Random Forest, Decision Tree, and Gradient Boosting. The Random Forest classifier significantly outperformed others, achieving a maximum accuracy of 99.4percents. The study concludes that the combination of fast static analysis and ML provides a robust, low-overhead solution for effectively classifying malware and benign executables. [Link](#)

3.2 Identifying Windows Installer Package Files for Detection of Pirated Software, By Dongjin Kim

This study addresses the difficulty of identifying pirated software distributed via Windows Installer package (.msi) files, as existing methods primarily target executables (PE). The authors propose an efficient software identification method based on extracting Globally Unique Identifiers (GUIDs) specifically the Upgrade Code, Product Code, and Package Code—directly from the MSI file’s metadata. They demonstrate that these GUIDs satisfy the criteria of efficiency, accuracy, and resistance to transformation attacks, as they are crucial for the legal installation and update process. Experiments show the method effectively identifies different applications, versions, and languages. Crucially, the extraction of GUIDs is significantly faster (around 0.9% of the time) than extracting IAT from embedded executables. [Link](#)

3.3 Malware Detection With Subspace Learning-Based One-Class Classification, By Hasan H. Al-Khshali

This research addresses the challenges of class imbalance and the curse of dimensionality in malware detection using machine learning. Recognizing that traditional supervised models misclassify evolving malware and suffer from heavily imbalanced datasets, the study proposes a novel pipeline leveraging Subspace Learning-based One-Class Classification (OCC) methods, specifically SSVDD and GESSVDD. OCC is adopted by training only on benign (non-malware) PE file features, which is highly effective against unknown malware. The core contribution is the joint optimization of a subspace and data description to mitigate high dimensionality while creating a tight boundary around the benign class. The proposed pipeline, evaluated on three PE-feature datasets, demonstrates highly promising results, notably achieving a True Positive Rate (TPR) of 100% for the subspace-learning-based OCC. [Link](#)

3.4 MSI - Masquerading as a Software Installer, By Cybereason Global SOC Team

This report, focused on the MSI file format abuse by threat actors, serves as an in-depth analysis of real-world malware families (e.g., Magniber Ransomware, MatanBuchus Loader, Qbot). It details the offensive (Red Team) and defensive (Blue Team) perspectives. The key takeaways are that malicious MSI files masquerade as legitimate installers, execute with elevated privileges (LocalSystem), and are exploited via Custom Actions and Sequence tables (specifically InstallExecuteSequence) to deploy payloads, often relying on misleading errors or suspicious installation paths (like LocalAppDataFolder) for stealth. The report uses manual analysis tools (Orca, msitools) to dissect the malicious logic, providing concrete indicators for static analysis and detection, which directly informs the feature engineering phase of a machine learning-based detection system. [Link](#)

3.5 Static Analysis for Malware Classification Using Machine and Deep Learning, By Marcelo Invert Palma Salas

This paper proposes a highly accurate malware classification methodology using static analysis of Portable Executable (PE) files. It evaluates seven Machine Learning algorithms and deep learning, demonstrating superior performance with LightGBM (Accuracy $\approx 100\%$, log loss 0.00118). The key contribution lies in the effective feature engineering phase, which uses N-grams (bytes and ASM Op-codes) and a SelectKBest technique to reduce the feature count from 84,611 to 893. This method, tested on the Microsoft Malware Classification Challenge dataset, achieves a real-time classification speed ($<2.3\text{ms}$), providing a crucial, state-of-the-art benchmark for highly accurate malware family identification based on PE file characteristics. [Link](#)

3.6 How to Analyze Malicious MSI Installer Files, By Nicole Fishbein (Israeli!) malware analyst

Nicole starts off by explaining how MSI files are trusted by both the users, and the Anti-virus defenders. The article details the internal structure of MSI files, including their use of relational databases with tables for components, features, files, directories, and actions. It explains how tools like Orca, msitools, lessmsi, and msidiff can be used by analysts to manually inspect and compare MSI files to detect malicious behavior. Real-world examples show how malware families like DarkGate, Maze Ransomware, IcedID, and SSLoad use MSI installers to hide payloads. The article also covers automated analysis methods (e.g., with Intezer) that detect reused malicious code within embedded binaries. Finally, it presents a case study of Iranian state sponsored group MuddyWater, which distributed MSI files disguised as legitimate software to install remote management tools like Atera Agent for espionage operations. [Link](#)

3.7 Malware Analysis and Detection Using Machine Learning Algorithms, By Tao Feng

This study addresses the growing threat of polymorphic malware; malicious software that constantly changes its code to evade detection. The authors propose a machine learning framework to detect these threats by analyzing features of the software. Traditional cybersecurity relies on signature-based detection, which compares files against a database of known virus "fingerprints." However, modern malware is "polymorphic," meaning it modifies its own signature traits to bypass defenses. To combat this, the researchers argue for Machine Learning approaches. ML algorithms can identify malicious patterns and behaviors, making them effective against new threats. Using over 17,000 samples from the Canadian Institute for Cybersecurity, researchers unpacked files in a virtual machine for analysis. They extracted and ranked key features to avoid overfitting, then trained supervised learning algorithms to accurately classify software as either benign or malicious. The research confirms that static analysis based on Machine Learning is a highly effective defense strategy. [Link](#)

3.8 Detecting emerging threats and reducing dwell time with machine learning, By Craig Chamberlain

The writer argues; Machine learning is incredibly fast at finding new cyber threats because it looks for unusual behavior rather than just known viruses. While standard security tools may take weeks to identify a new threat, ML can spot it in minutes. The writer gives an MSI file example, relevant to our research: "A dangerous virus targeting Ukraine was hidden inside a Windows installer file (.msi). Because it looked like a normal system file, standard antivirus software ignored it for 46 days." How ML Caught It: The ML system detected the virus in just 16 minutes by spotting two anomalies: Strange Network Activity: The virus pretended to be a standard Windows process (sppsvc.exe), but it tried to connect to the internet. The real Windows process almost never does this. Weird File Usage: The virus loaded internal files (DLLs) that the real process does not use. Because the software was behaving differently than a normal computer should, the ML system flagged it immediately, reducing the time the virus was active from 46 days to less than an hour. [Link](#)

3.9 Analyzing MSI files, By Didier Stevens

Security researcher Didier Stevens explains how to inspect MSI files to find hidden malware. He notes that MSI files are actually OLE (Object Linking and Embedding) files, meaning they act like containers that can hold other files inside them. The Approach: Stevens uses a tool called oledump combined with YARA rules to scan the MSI file. This scans the "streams" internal data lanes of the file to look for hidden executables. In his example, he found two suspicious items: A CAB file (a compressed archive) containing an executable. A standalone PE file (executable). When looking at the internal file structure, the names of the streams were unreadable (encoded). To fix this, Stevens developed a custom plugin for his tool (pluginmsi). The Result: The plugin decoded the text to reveal the name Binary.bz.CustomAction. This confirmed that the hidden file was a malicious DLL (Dynamic Link Library). With this article we can learn and adapt on how to approach MSi files manually, which will later maybe help us to compare to our ML way of things. [Link](#)

3.10 Features of Detecting Malicious Installation Files Using Machine Learning Methods, By P.E.Yugai

The paper investigates the use of machine learning algorithms to detect malicious installation files, particularly Trojan installers and downloaders. It performs a comparative analysis of several algorithms, including Naive Bayes, Random Forest, and C4.5, implemented using Weka. The study identifies the most significant features that distinguish legitimate installation files from malicious ones, demonstrating that machine learning can effectively classify and detect Trojan-type installers. [Link](#)

Article	Focus	Approach	Contribution to Our Research
Malware Detection in PE files using Machine Learning (Samarth Tyagi)	Static analysis of PE files for malware detection.	Extraction of PE header features and supervised ML models (Random Forest, Decision Tree, Gradient Boosting) and feature reduction from 54 to 13	Identifies key static features for ML malware detection and shows that Random Forest achieves high accuracy useful guidance for our MSI based feature extraction
Identifying Windows Installer Package Files for Detecting Pirated Software (Dongjin Kim)	Detecting pirated software distributed via MSI files.	Uses three MSI GUIDs(Upgrade, Product, Package) as fast, unique, and tamper-resistant identifiers	Shows MSI-native meta-data is a strong static feature source, useful for ML-based detection of malicious or modified MSI installers
Malware Detection With Subspace Learning-Based One-Class Classification (Hasan Al-Khshali)	Malware detection in PE files under strong class imbalance and evolving/unknown malware	Uses subspace-learning one-class classifiers (OCSVM, SVDD, SSVDD, GESSVDD) trained only on benign PE-header features; evaluated on three datasets and achieves TPR up to 100%	Motivates using one-class / anomaly detection on MSI features and shows the importance of handling class imbalance and the curse of dimensionality in our MSI-based ML model
MSI – Masquerading as a Software Installer (Cybereason SOC)	MSI abuse by malware families (Magniber, MatanBuchus, Qbot)	Red/Blue/Purple-Team analysis of MSI internals via Orca, msitools, ole-dump	Indicators (fake errors, Local App Data Folder, signature mismatch, async Custom Actions) supporting our ML oriented MSI feature selection
Static Analysis for Malware Classification Using Machine and Deep Learning (Marcelo Salas)	Static PE malware classification	N-grams and SelectKBest (84k to 893) with 7 ML models, LightGBM =100percent accuracy and smaller than 2.3ms inference	Strong static ML pipeline useful to guide MSI feature engineering and model choice
How to Analyze Malicious MSI Installer Files (Nicole Fishbein)	Analysis of MSI malware	Examination of streams, usable software, and embedded scripts	Provides specific MSI indicators usable for our dataset
Malware Analysis and Detection Using Machine Learning Algorithms (Tao Feng)	General malware detection using ML	ML models resilient to code "Polymorphic" code	Reinforces the effectiveness of ML for detecting malicious installer payloads
Detecting Emerging Threats With Machine Learning (Craig Chamberlain)	Broad ML-based threat detection	Anomaly detection using ML	Highlights the value of using anomaly based analysis in MSI detection
Analyzing MSI Files (Dieder Stevens)	Internal structure and analysis of MSI files	Low-level technical inspection tools	Helps us to understand infected MSI's
Features of Detecting Malicious Installation Files Using ML Methods (Yugai)	ML detection of Trojan installers	Using ML algorithms: NBC, Random Forest, C4.5 in Weka	Helps identify the most relevant ML models

4 Methodology

The proposed methodology focuses on developing a machine-learning-based detection model for identifying malicious MSI installation files using static analysis. Our approach consists of four main stages:

Feature Extraction from MSI Files We extract static attributes from MSI packages, including metadata fields, internal streams, custom actions, embedded scripts, and structural properties. These features are selected based on known MSI attack patterns and findings from the literature.

Dataset Construction and Labeling A dataset of both benign and malicious MSI files is collected. Files are labeled according to verified threat intelligence sources, sandbox results, and manual inspection to ensure accuracy and reduce noise in the training set.

Model Development Using Machine Learning Multiple machine-learning classifiers (e.g., Random Forest, C4.5, Naive Bayes, or additional models as needed) are trained on the extracted features. A comparison between classification methods is conducted to determine the most effective approach for MSI malware detection.

Evaluation and Validation The model is evaluated using cross-validation, accuracy metrics, ROC analysis, and confusion matrices. Both false positives and false negatives are examined to refine the feature set and improve the model’s reliability.

This methodology is designed to create an efficient, lightweight, and static-analysis-based system capable of detecting malicious installers before execution, reducing the risk posed by MSI based malware distribution.

5 Dataset Design and Specification

5.1 Data Sources and Size

The dataset will be sourced from two main sources: Malicious Samples will be Collected from advanced threat intelligence platforms and repositories, specifically from MalwareBazaar and VirusShare. Benign Samples: Gathered from legitimate, trusted software vendors (such as Microsoft and Adobe) and standard, verified MSI installation packages found within the Windows operating system environment.

5.2 Feature Engineering and Labeling

The model will be trained on extracted features rather than the raw binary files. A set of 80 to 150 features will be engineered, categorized as follows:

Static/Structural Features: Derived from internal analysis of the MSI file’s OLE structure. These include quantitative metrics such as file size, counts of critical tables (e.g., CustomAction, Binary), and analysis of encoded strings or suspicious stream names .**Behavioral Features** will be collected by executing the MSI file in an isolated virtual environment. Features will capture suspicious activities, including anomalous API calls, Registry modifications, and unusual network communication attempts. **Dependency/Reference Features:** Analysis focusing on external calls, such as required DLLs, and the invocation of scripting environments via the CustomAction mechanism.

Each sample will be assigned a binary label: **1** (Malicious) or **0** (Benign).

5.3 Splitting Strategy

The finalized, feature-extracted dataset will be subjected to a random split to ensure uniform feature distribution across all subsets used in the modeling pipeline:

- **Training Set:** 75% of the data, used for the initial model training with 5 folds.
- **Testing Set:** 25% of the data, reserved for the final, unbiased evaluation of the model’s performance.

6 Comparative Analysis and Proposed MSI Malware Detection Approach

Experimental setup and dataset consistency. To ensure a fair and meaningful comparison between all evaluated approaches, all models are trained and tested on the exact same set of MSI files. A single dataset of benign and malicious installers is constructed, and a fixed train-test split (75%

/ 25%) is applied consistently across all experiments. The dataset is balanced because we have 100 benign and 100 malicious msi files.

For each method (Tyagi-based, Yugai-based, baseline MSI-only, and the proposed approach), features are extracted from the same MSI samples, labels are kept identical, and the same classification algorithm and hyperparameters are used when applicable. Consequently, any observed performance differences can be attributed solely to the feature representations and modeling choices, rather than to variations in the dataset or evaluation protocol.

6.1 Tyagi approach (article 1)

Tyagi et al. propose a static malware detection approach based on structural analysis of Portable Executable (PE) files and supervised machine learning. Their method relies on PE header fields (e.g., *Machine*, *Subsystem*, *DllCharacteristics*) and entropy-based features computed on PE sections and resources, with Random Forest reported as the most effective classifier.

Adaptation and insights. To apply this approach in the MSI context, we extract all PE payloads embedded in each MSI package and aggregate the features at the MSI level. Entropy-based features are summarized using statistical measures, while categorical PE header attributes are aggregated using the mode. This adaptation highlights that structural PE features and the number of embedded executables provide strong static signals. However, this approach does not model the installer logic itself, which motivated the inclusion of MSI-level behavioral features in our proposed method.

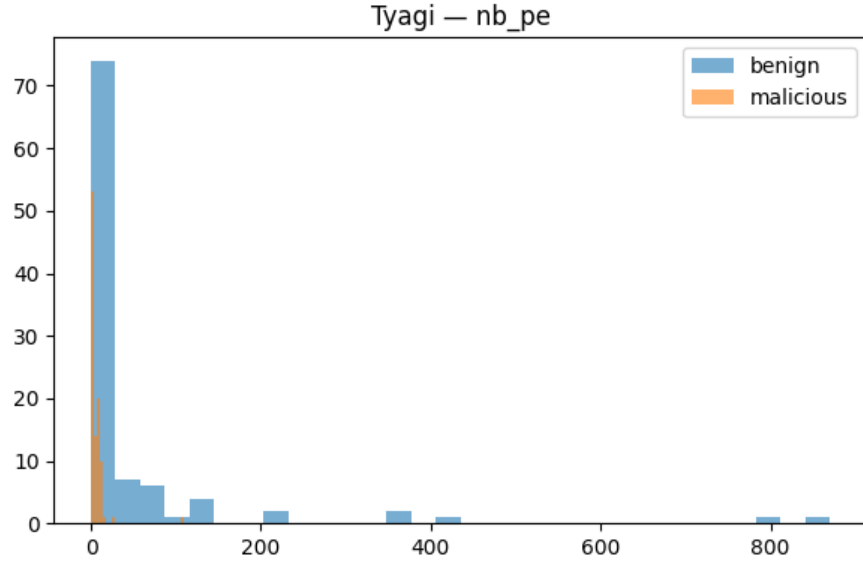


Figure 1: Distribution of the number of embedded PE files per MSI package (`nb_pe`) using the Tyagi feature set. Benign installers typically contain more PE files than malicious ones.

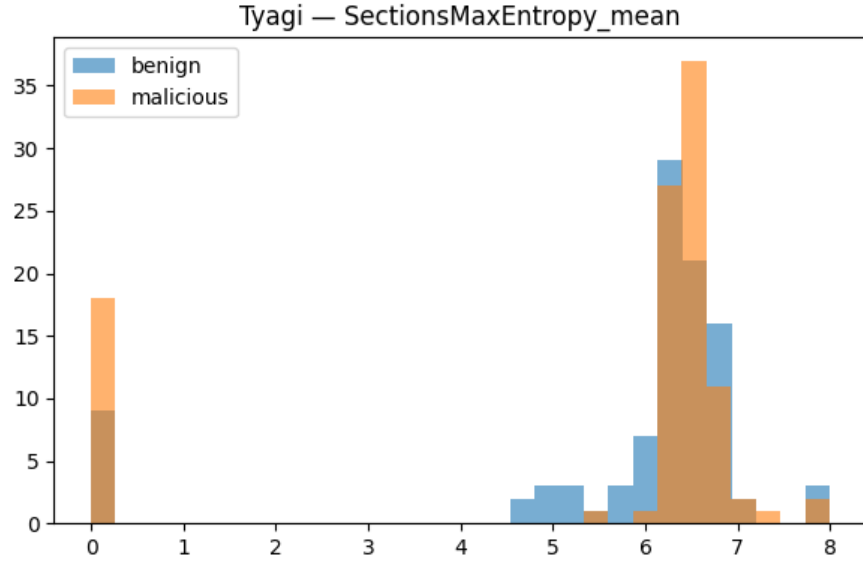


Figure 2: Distribution of the mean maximum section entropy for benign and malicious MSI packages. Higher entropy values are more frequent in malicious samples, suggesting obfuscation or packing.

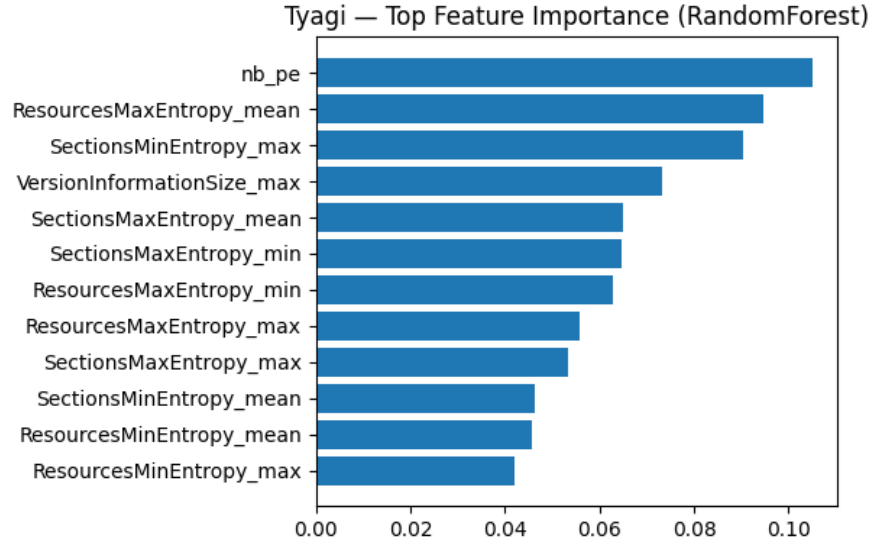


Figure 3: Top feature importance obtained using a Random Forest classifier on the Tyagi feature set. The number of embedded PE files (`nb_pe`) is the most discriminative feature.

Table 1: Classification performance of the Tyagi feature set using a Random Forest classifier on the holdout test set.

Method	Model	Accuracy	Precision (Mal.)	Recall (Mal.)	F1-score (Mal.)
Tyagi	Random Forest	0.76	0.74	0.80	0.77

6.2 Yugai approach (article 2)

Yugai et al. propose a static detection method for malicious installer files based on lightweight Portable Executable (PE) metadata and resource-related features. Their approach focuses on the presence of PE resources (e.g., icons, manifests, version information), basic structural attributes such as the number of sections, and metadata fields extracted from the version information. Several classifiers are evaluated, including Naive Bayes, Decision Tree (C4.5), and Random Forest.

Adaptation and insights. To adapt this method to MSI packages, we extract all embedded PE payloads from each installer and aggregate the Yugai features at the MSI level. Binary indicators are combined using a logical maximum, while numeric attributes such as the number of sections are summarized using their maximum value. This adaptation preserves the original feature semantics while enabling MSI-level classification.

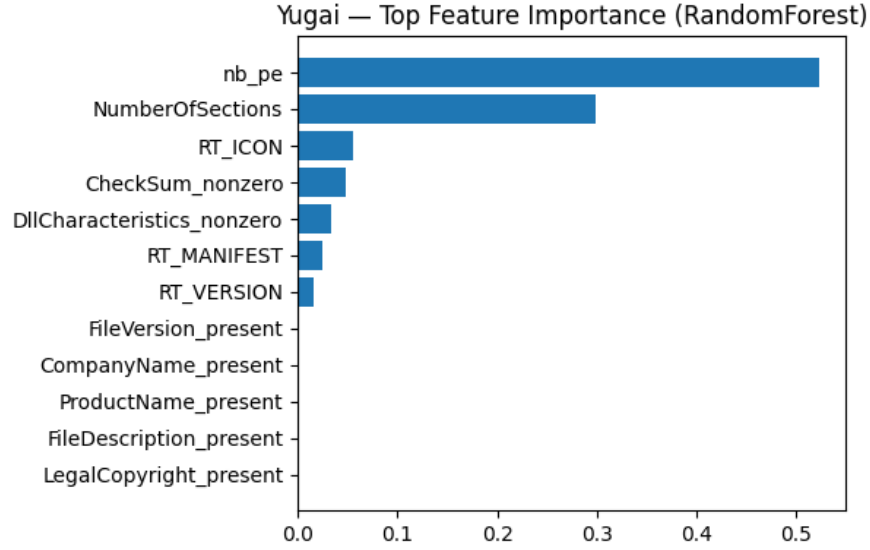


Figure 4: Top feature importance obtained using a Random Forest classifier on the Yugai feature set. The number of embedded PE files (`nb_pe`) and the number of sections are the most discriminative features.

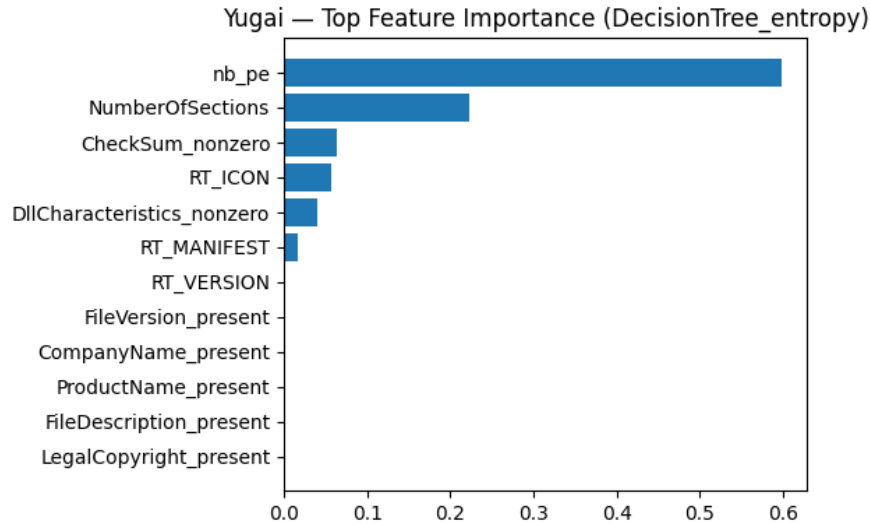


Figure 5: Top feature importance obtained using a Decision Tree (entropy criterion) on the Yugai feature set. Similar to Random Forest, structural PE features dominate the decision process.

Model comparison. Consistent with the findings of Yugai et al., tree-based classifiers outperform probabilistic models on this feature set. Random Forest achieves the best overall performance, while Naive Bayes performs close to random guessing, as confirmed by ROC and precision–recall curves.

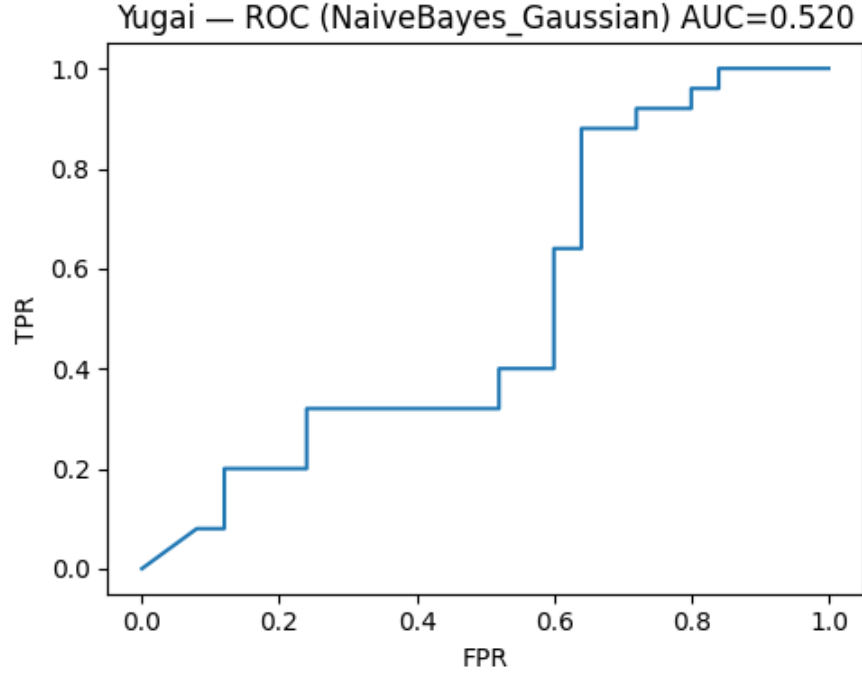


Figure 6: ROC curve obtained using a Gaussian Naive Bayes classifier on the Yugai feature set. The near-random AUC indicates limited discriminative power for this model.

Table 2: Classification performance of the Yugai feature set using different classifiers on the holdout test set.

Model	Accuracy	Precision (Mal.)	Recall (Mal.)	F1-score (Mal.)	ROC AUC
Random Forest	0.76	0.71	0.88	0.79	0.82
Decision Tree (Entropy)	0.74	0.71	0.80	0.75	0.77
Naive Bayes	0.58	0.54	1.00	0.70	0.52

Limitations. While the Yugai feature set captures meaningful structural differences between benign and malicious installers, it remains limited to shallow PE metadata. The absence of installer-level behavioral information motivates the integration of MSI-specific features in the proposed method.

6.3 Baseline MSI-only vs. Proposed MSI + PE-structure Model

To evaluate the contribution of PE structural information, we first establish a baseline model relying exclusively on MSI-level features. This baseline excludes all PE-derived characteristics and focuses on installer metadata, execution flow, and custom action behavior.

Baseline (MSI-only). The baseline model is trained using a Random Forest classifier on MSI-specific features, including file size, header entropy, presence of binary tables, execution sequence statistics, and custom action properties. As shown in Table 3, this model already achieves strong performance, confirming that installer-level information captures relevant malicious behaviors.

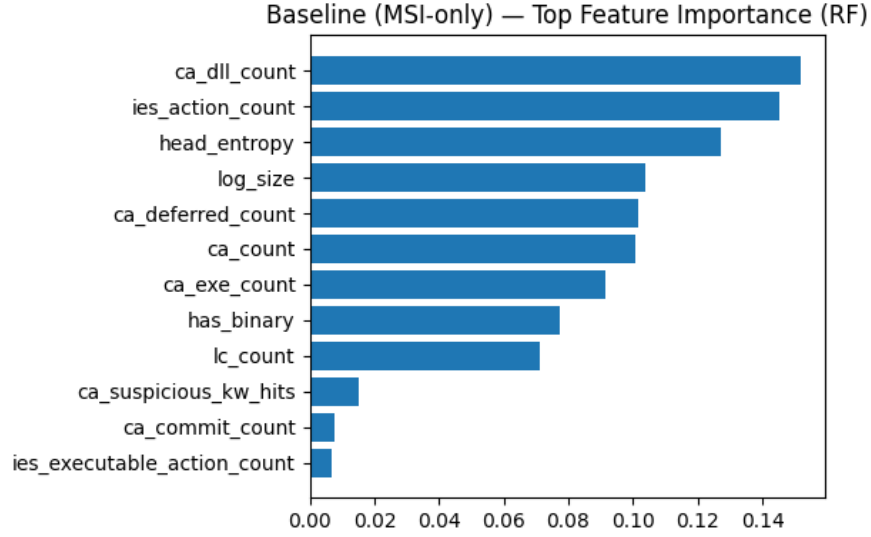


Figure 7: Top feature importance for the baseline MSI-only Random Forest model. Custom action counts, execution sequence statistics, and header entropy are the most influential features.

Proposed model (MSI + PE-structure). We then extend the baseline by incorporating lightweight PE structural features extracted from embedded payloads, namely the total number of PE files, executables, and DLLs. These features are inspired by static PE-based approaches, while remaining compatible with an MSI-level analysis.

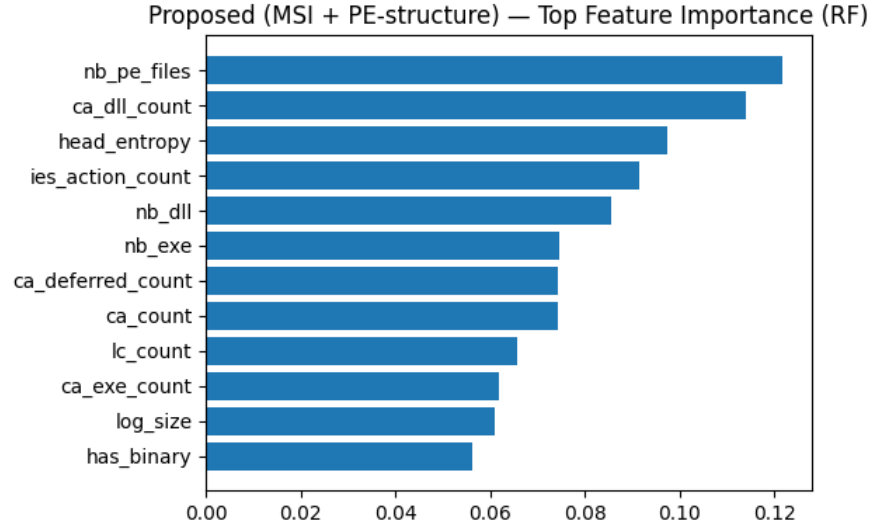


Figure 8: Top feature importance for the proposed Random Forest model combining MSI-level and PE structural features. The number of embedded PE files becomes the most discriminative feature.

Performance comparison. Table 3 summarizes the holdout test results. The proposed model improves overall accuracy from 0.92 to 0.94 while preserving a high recall for malicious samples. The confusion matrices indicate a reduction in false positives without increasing false negatives.

Table 3: Performance comparison between the baseline MSI-only model and the proposed MSI + PE-structure model using Random Forest.

Method	Accuracy	Precision (Mal.)	Recall (Mal.)	F1-score (Mal.)
Baseline (MSI-only)	0.92	0.89	0.96	0.92
Proposed (MSI + PE)	0.94	0.92	0.96	0.94

Discussion. These results demonstrate that MSI-level behavioral features already provide strong detection capabilities. However, adding simple PE structural signals significantly improves discrimination, particularly by capturing differences in payload composition. This confirms that the proposed approach effectively combines the complementary strengths of installer behavior analysis and static PE structure inspection.

6.4 Extended Model Comparison on the Proposed Feature Set

To further validate the robustness of the proposed MSI malware detection approach, we evaluated multiple machine learning classifiers using the same feature set (MSI-level behavioral features combined with PE structural information) and the same experimental protocol. All models were trained and tested on the identical dataset and train-test split to ensure a fair comparison.

Model performance analysis. Table 4 summarizes the classification performance of several representative model families. Ensemble tree-based methods clearly outperform linear, probabilistic, and distance-based classifiers. Random Forest and Gradient Boosted Decision Trees (GBDT) achieve the highest overall accuracy (0.94), significantly exceeding Logistic Regression, Support Vector Machines, and k-Nearest Neighbors.

Table 4: Performance comparison of different classifiers using the proposed MSI + PE-structure feature set.

Model	Accuracy	Precision (Mal.)	Recall (Mal.)	F1-score (Mal.)
Random Forest	0.94	0.92	0.96	0.94
Gradient Boosted DT	0.94	0.89	1.00	0.94
Extra Trees	0.90	0.88	0.92	0.90
k-Nearest Neighbors	0.76	0.69	0.96	0.80
Logistic Regression	0.72	0.67	0.88	0.76
Support Vector Machine	0.54	0.52	1.00	0.68

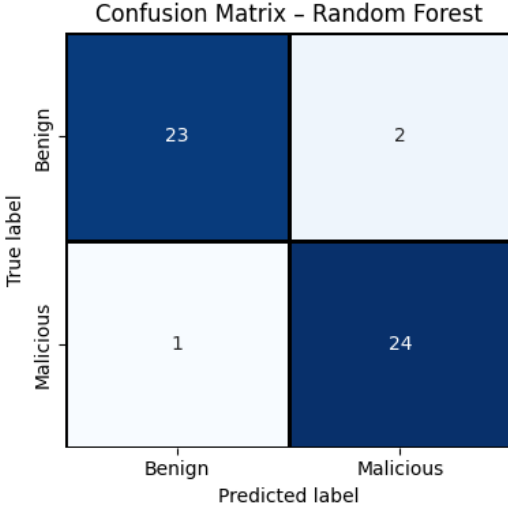


Figure 9: *
(a) Random Forest

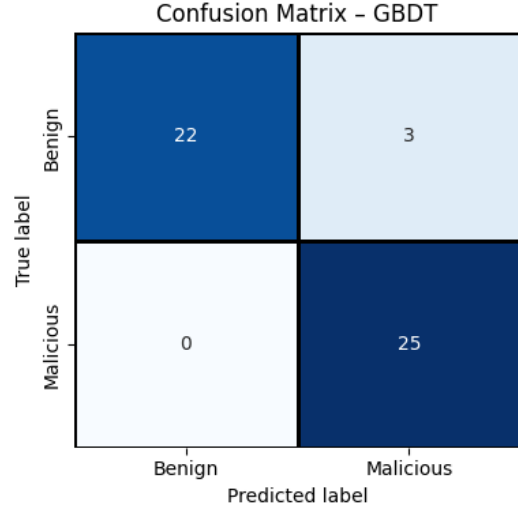


Figure 10: *
(b) Gradient Boosted Decision Trees

Figure 11: Confusion matrices of the two best-performing models (Random Forest and GBDT) evaluated on the proposed MSI + PE-structure feature set.

Model selection rationale. Although Random Forest and Gradient Boosted Decision Trees achieve the same overall accuracy (0.94) on the holdout test set, their error profiles differ. GBDT attains perfect recall for malicious samples, eliminating false negatives, but introduces a higher number of false positives. In contrast, Random Forest provides a more balanced trade-off between precision and recall, reducing false alarms while maintaining a high malicious recall (0.96).

In practical deployment scenarios, excessive false positives may significantly impact usability and operational costs, as benign installers could be incorrectly flagged as malicious. For this reason, Random Forest was selected as the final model, as it offers a more stable and interpretable decision process, stronger robustness on small datasets, and a lower false positive rate while preserving strong detection capabilities.

7 Conclusion

In this phase, we validated that our proposed MSI malware detection approach effectively combines the complementary strengths of two static-analysis research directions. Inspired by Yugai and Tyagi, we integrated lightweight PE structural signals (number of embedded PE files, executables, and DLLs) into an MSI-focused feature set capturing installer logic through sequence tables and custom actions. Using a strictly controlled experimental protocol where all models were trained and tested on the exact same MSI samples with an identical train-test split, we observed a consistent improvement when adding PE-structure features. In particular, the proposed model improved holdout accuracy from 0.92 (MSI-only baseline) to 0.94, while maintaining a high recall for malicious installers (0.96) and increasing precision for the malicious class. Feature importance analysis further confirms this synergy: MSI behavioral indicators remain highly influential, while the PE composition feature (`nb_pe_files`) becomes the strongest discriminator, capturing payload structure differences that MSI-only features cannot fully represent.

Overall, these results support the central claim of our work: MSI-level behavioral characterization is already a strong detection signal, and augmenting it with simple PE structural features yields a more robust and accurate static classifier for malicious MSI installers.